TRAFFIQ 2025

AI Model Technical Report

Autonomous Vehicle Navigation using Behavioral Cloning

IET On-Campus JU × JUMPER
Jadavpur University
April 2025

1. Executive Summary

This report describes our end-to-end AI system for autonomous vehicle navigation developed for the TRAFFIQ competition. Our approach is based on Behavioral Cloning (BC), a supervised imitation learning technique in which a deep convolutional neural network learns to map raw camera images directly to steering commands, replicating the behaviour of a human expert driver.

The model is based on the well-established NVIDIA Dave-2 (End-to-End) architecture, trained on the Udacity Behavioural Cloning dataset, and exported to ONNX format for efficient real-time inference on the competition-provided Raspberry Pi 4B 8GB hardware. The complete pipeline from camera capture to motor control runs locally on the Pi with no external communication.

2. System Overview

2.1 High-Level Pipeline

The autonomous driving pipeline consists of five sequential stages:

- Camera Frame Capture — Raspberry Pi Camera V2 streams 30 FPS video.
- Preprocessing — Frames are converted to YUV colour space, cropped, and resized to match the training distribution exactly.
- Model Inference — The preprocessed frame is fed into the ONNX model running on ONNX Runtime (ARM-optimised). The model outputs a single steering angle value in the range [-1, +1].
- Control Mapping — The predicted steering angle is mapped to a PWM pulse width sent to the steering servo.
- Actuation — A fixed-throttle PWM signal drives the ESC/motor; steering PWM drives the servo.

2.2 Hardware Platform

Component	Specification	Role
Raspberry Pi 4B	8 GB RAM, ARM Cortex-A72	Main compute unit — model inference + control
Raspberry Pi Camera V2	Sony IMX219, 8 MP, 30 FPS	Real-time visual input
Steering Servo	Standard PWM servo (1000–2000 µs)	Lane-tracking steering control
ESC + DC Motor	Standard RC car ESC	Throttle / forward motion
GPIO (pigpio)	Hardware PWM, GPIO 18 & 19	Precise pulse-width modulation

3. Dataset

3.1 Source

We used the Udacity Self-Driving Car Behavioural Cloning Dataset (Jungle Track variant), publicly available on Kaggle. The dataset was recorded from a simulated driving environment using the Udacity Car Simulator.

3.2 Dataset Statistics

Property	Value
Total Samples (rows)	3,404
Training Set (80%)	2,723 samples
Validation Set (10%)	340 samples
Test Set (10%)	341 samples
Input Columns	center, left, right camera image paths
Target Column	steering angle (float, range ~[-1, +1])
Additional Columns	throttle, brake, speed
Image Resolution (raw)	320 × 160 pixels, RGB

3.3 Data Loading & Path Fixing

The original CSV was recorded on a Windows machine with backslash-separated paths. A path-fixing function was applied to normalise all image paths to the Linux filesystem:

```
def path_leaf(path):
  head, tail = ntpath.split(path)
  return tail
data['center'] = data['center'].apply(path_leaf)
data['left'] = data['left'].apply(path_leaf)
data['right'] = data['right'].apply(path_leaf)
data.head()
```

After fixing, 100% of image paths were validated as accessible on disk.

4. Data Preprocessing & Augmentation

4.1 Multi-Camera Strategy

The Udacity simulator records three simultaneous camera feeds: centre, left, and right. During training, a camera is randomly selected for each sample. A steering correction of +0.2 is applied for left-camera images (car should steer right to re-centre) and -0.2 for right-camera images. This effectively triples the usable dataset size and teaches the model recovery behaviour.

Camera	Steering Correction	Purpose
Centre	0.0 (none)	Primary driving line
Left	+0.2	Teaches rightward correction
Right	–0.2	Teaches leftward correction

4.2 Image Preprocessing Pipeline

→ The image preprocessing pipeline transforms raw camera images into a format suitable for the model.

→ First, images are converted from BGR to RGB format and cropped to remove irrelevant regions such as the sky and vehicle hood, focusing only on the road. The cropped images are then resized to a fixed resolution of 66×200 pixels to match the model input.

→ Pixel values are normalized to the range [0,1] to improve training stability. Additionally, data augmentation techniques such as random brightness adjustment, shadow addition, flipping, and translation are applied to increase dataset diversity and improve robustness under varying lighting and driving conditions.

The preprocessing in code:

```
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
img = cv2.cvtColor(img, cv2.COLOR_RGB2YUV)
img = img[60:-25, :, :]   # crop sky and hood
img = cv2.resize(img, (200, 66))
```

5. Model Architecture

5.1 NVIDIA Dave-2 (End-to-End CNN)

The model architecture is based on the end-to-end learning approach proposed by NVIDIA for self-driving cars. It consists of a series of convolutional layers followed by fully connected layers.

The convolutional layers automatically extract spatial features such as lane markings and road edges, while the fully connected layers map these features to a steering angle output.

The model takes a 3-channel image as input and produces a single continuous value representing the steering command. Activation functions such as ReLU are used to introduce non-linearity, enabling the model to learn complex driving patterns.

5.2 Layer-by-Layer Architecture

Layer #	Type	Configuration	Output Shape
Input	Input	Shape (66, 200, 3)	(None, 66, 200, 3)
L1	Lambda (Normalisation)	x / 255.0	(None, 66, 200, 3)
L2	Conv2D	24 filters, 5×5, stride 2×2, ReLU	(None, 31, 98, 24)
L3	Conv2D	36 filters, 5×5, stride 2×2, ReLU	(None, 14, 47, 36)
L4	Conv2D	48 filters, 5×5, stride 2×2, ReLU	(None, 5, 22, 48)
L5	Conv2D	64 filters, 3×3, stride 1×1, ReLU	(None, 3, 20, 64)
L6	Conv2D	64 filters, 3×3, stride 1×1, ReLU	(None, 1, 18, 64)
L7	Flatten	—	(None, 1152)
L8	Dense	100 units, ReLU	(None, 100)
L9	Dense	50 units, ReLU	(None, 50)
L10	Dense	10 units, ReLU	(None, 10)
L11	Dense (Output)	1 unit, Linear	(None, 1)

5.3 Design Rationale

- The system is designed to perform end-to-end learning, directly mapping input images to steering angles without explicit feature engineering. This approach simplifies the pipeline and allows the model to learn relevant features automatically.
- Data balancing and augmentation are incorporated to address biases such as over-representation of straight driving and to improve performance in diverse conditions, including low light.
- Transfer learning and fine-tuning are used to extend the model's capability to handle new scenarios such as obstacle avoidance while preserving previously learned lane-following behavior.
- The overall design prioritizes efficiency, making it suitable for deployment on resource-constrained devices like the Raspberry Pi.

5.4 Training Configuration

Hyperparameter	Value	Justification
Batch Size	64	Provides a good balance between training stability and computational efficiency.
Learning Rate (initial)	1e-4	Allows stable learning without large oscillations in loss.
Learning Rate (fine-tuning)	1e-5	Prevents drastic weight updates and preserves learned features during fine-tuning.
Optimizer	Adam	Efficient optimization with adaptive learning rates, suitable for deep networks.
Loss Function	Weighted MSE Loss	Emphasizes larger steering angles to improve performance on sharp turns.
Number of Epochs	40–50	Ensures sufficient training while avoiding overfitting.
Input Image Size	(3, 66, 200)	Matches NVIDIA model architecture and reduces computational load.
Data Augmentation	Brightness, Shadow, Flip, Translation	Improves generalization and robustness under varying conditions.
Steering Correction (Left/Right Cameras)	±0.3	Helps model learn recovery behavior when the vehicle deviates from center.
Dropout	Not used	Model complexity is moderate; regularization handled via augmentation.
Activation Function	ReLU	Introduces non-linearity and avoids vanishing gradient problem.
Output Activation	Tanh	Restricts steering output to range [-1, 1] for stable predictions.
Dataset Balancing	Reduced straight, boosted turns	Prevents bias toward straight driving and improves turning accuracy.

6. Training Result

6.1 Loss Curves

The model was trained for 20 epochs. Both training loss (MSE) and validation loss converged steadily, with no significant overfitting observed.

Epoch	Train Loss (MSE)	Val Loss (MSE)	Train MAE	Val MAE
1	0.1832	0.1241	0.3218	0.2640
5	0.1202	0.1098	0.2655	0.2529
10	0.0998	0.0970	0.2409	0.2285
15	0.0844	0.0898	0.2246	0.2254
16	0.0835	0.0830	0.2189	0.2157

Epoch	Train Loss (MSE)	Val Loss (MSE)	Train MAE	Val MAE
20	0.0745	0.0809	0.2071	0.2132

6.2 Observations

- The best validation loss of 0.0809 MSE was achieved at Epoch 20, with validation MAE of 0.2132 (approximately 12.7 degrees on a [-1,+1] normalised scale).
- Training and validation curves track closely, indicating the model has not overfit to the training data.
- The multi-camera augmentation strategy with steering correction effectively regularised the model and improved recovery behaviour.
- MAE of ~0.21 corresponds to an average steering error of roughly 4.7 degrees in real-world terms — sufficient for smooth lane-following at moderate speeds.

7. Model Export & Deployment Format

7.1 Pytorch to ONNX Conversion

The trained model was converted from PyTorch to ONNX format to enable deployment on the Raspberry Pi. ONNX provides a framework-independent representation for efficient inference.

The conversion was performed using $torch.onnx.export(\quad)$ with a dummy input of shape (1, 3, 66, 200), matching the model's input. The model was set to evaluation mode to ensure consistent behavior.

The exported ONNX model was then used with ONNX Runtime for fast and efficient inference on CPU-based systems.

7.2 Runtime Inference on Raspberry Pi

Inference is performed using onnxruntime:

```
import onnxruntime as ort
session = ort.InferenceSession('self_driving_model.onnx')
input_name = session.get_inputs()[0].name
steering = session.run(None, {input_name: frame})[0][0][0]
```

Expected inference latency on RPi 4B: approximately 50–100 ms per frame, yielding 10–20 FPS — adequate for the arena speeds expected.

8. Real-Time Drive Script (drive.py)

8.1 Camera Interface

The Raspberry Pi Camera V2 is accessed via OpenCV's VideoCapture interface at /dev/video0. Frames are captured at 320×240 resolution before being downscaled by the preprocessing pipeline.

8.2 Steering Angle to PWM Mapping

The model output (range approximately -1.0 to +1.0) is mapped to a servo PWM pulse width:

```
pwm = int(1500 + steering_angle * 500)
pwm = clip(pwm, 1000, 2000)   # microseconds
```

Steering Angle	PWM (µs)	Physical Meaning
-1.0 (full left)	1000	Maximum left lock
0.0 (straight)	1500	Centre / straight ahead
+1.0 (full right)	2000	Maximum right lock

8.3 Safe-Stop Mechanism

As required by the TRAFFIQ rules, the system implements a safe-stop function that is triggered on camera failure, KeyboardInterrupt, or unexpected exceptions. On activation it immediately sets throttle to neutral (1500 µs) and steering to centre (1500 µs) before releasing GPIO resources.

9. Compliance with TRAFFIQ Rules

Rule	Our Implementation
Fully autonomous once started	Run loop executes without any human input after launch
No hardcoded coordinates or path scripts	Navigation is 100% based on real-time camera inference
No external communication during run	All computation runs locally on RPi; WiFi/BT disabled
Safe-stop mechanism	Implemented in finally block; triggers on any failure
Approved hardware only	Computation runs entirely on provided RPi 4B
Robust to lighting changes	YUV colour space separates luminance; model trained on varied conditions
Own model / credited open source	NVIDIA Dave-2 architecture (cited); Udacity dataset (cited); all code original
Model inference within latency threshold	ONNX Runtime ARM-optimised; ~50–100 ms per frame

10. Limitations & Planned Improvements

10.1 Known Limitations

- Sim-to-Real Gap: The model was trained on a simulator dataset. Real-world camera images will differ in colour profile, lighting, and road texture. Fine-tuning on arena footage (if permitted) would significantly improve performance.
- Fixed Throttle: Speed is currently constant. Dynamic speed control based on curvature or obstacle proximity is not yet implemented.

- No Obstacle Avoidance: The current model predicts only steering; no object detection module is integrated for obstacle avoidance.
- Crop Sensitivity: The hardcoded crop [60:-25] assumes the training camera height. Adjustments may be required based on the actual RPi camera mounting position on the competition car.

10.2 Planned Improvements for Final Round

- Camera calibration and histogram equalisation to bridge the sim-to-real colour gap.
- Adaptive throttle control: reduce speed proportionally when |steering_angle| > 0.3.
- Integration of a lightweight YOLOv5n or MobileNet-SSD object detector for obstacle detection as a second inference pass.
- Real-time frame capture using picamera2 library for lower latency on Raspberry Pi OS Bookworm.

11. Technology Stack

Component	Technology	Version
Deep Learning Framework	Pytorch	2.x
Model Architecture	NVIDIA Dave-2 (Custom Keras Implementation)	—
Training Dataset	Udacity Behavioural Cloning (Jungle Track)	—
Model Export	`torch.onnx.export()`	1.17.0
Inference Runtime	ONNX Runtime (ARM)	Latest
Computer Vision	OpenCV (cv2)	4.x
GPIO Control	pigpio	Latest
Data Processing	NumPy, Pandas	Latest
Visualisation	Matplotlib, Seaborn	Latest
Edge Hardware	Raspberry Pi 4B 8GB	—
Camera	Raspberry Pi Camera Module V2	—

12. Obstacle Detection Module (YOLOv8n)

12.1 Overview

In addition to the core behavioral cloning steering model, we have integrated a real-time obstacle detection module using YOLOv8 Nano (YOLOv8n) — the smallest and fastest variant in the YOLOv8 family by Ultralytics. This module processes the same camera feed in parallel and triggers evasive or braking behaviour when an obstacle is detected within a defined proximity threshold.

12.2 Dataset

The obstacle detection model was trained on the Udacity Self-Driving Car Obstacles Dataset, downloaded from Kaggle (barkataliarbab/udacity-self-driving-car-obstacles-dataset). The dataset is structured in YOLO format:

Component	Details
Dataset Source	Kaggle: barkataliarbab/udacity-self-driving-car-obstacles-dataset
Format	YOLO format (class cx cy w h, normalised coordinates)
Directory Structure	export/images/ and export/labels/
Config File	data.yaml (class names, train/val/test paths)
Label Format Example	1 0.104 0.55 0.208 0.273 (class x_center y_center width height)

Sample label entries observed in the dataset:

```
1 0.1049 0.55   0.2088 0.2733
1 0.2973 0.5233 0.0406 0.0633
1 0.3354 0.5325 0.0520 0.065
1 0.3770 0.5283 0.0354 0.0666
1 0.4223 0.5533 0.1010 0.1266
```

12.3 Model: YOLOv8 Nano

YOLOv8n (Nano) was selected over larger variants for two reasons: inference speed on the Raspberry Pi 4B, and the relatively simple detection task (detecting obstacles in a bounded arena). YOLOv8n achieves real-time performance on edge hardware while maintaining sufficient accuracy for this use case.

Property	Value
Model	YOLOv8n (Nano)
Framework	Ultralytics 8.4.34
Base Weights	yolov8n.pt (COCO pretrained)
Task	Object Detection (single class: obstacle)
Input Size	640 × 640 pixels
Training Epochs	50
Batch Size	16 (default)
Augmentation	Default Ultralytics augmentation pipeline
Backend	PyTorch 2.1.0 + CPU (Intel Xeon, Colab training)

12.4 Training Code

The model was trained using the Ultralytics high-level training API:

```
from ultralytics import YOLO
```

```
model = YOLO('yolov8n.pt')
model.train(
    data=os.path.join(path, 'data.yaml'),
    epochs=50,
    imgsz=640
)
```

Note: During development, two errors were encountered and resolved. First, a shutil.copytree KeyboardInterrupt occurred when attempting to copy the read-only Kaggle dataset to a working directory — resolved by training directly from the Kaggle input path. Second, an OSError (Errno 30 — read-only filesystem) occurred when trying to create subdirectories inside /kaggle/input — resolved by pointing data.yaml paths to the original export/ structure rather than a reorganised copy.

12.5 Integration with Drive Pipeline

On the Raspberry Pi, YOLOv8n inference runs as a second pass on the same captured frame, immediately after the steering model inference. The detection result is used to override normal driving behaviour:

Condition	Action
No obstacle detected (confidence < threshold)	Normal steering from behavioral cloning model
Obstacle detected, bounding box small (far)	Slight steering correction away from obstacle centre
Obstacle detected, bounding box large (close)	Reduce throttle; increase steering correction
Obstacle fills >30% of frame width	Emergency stop (safe-stop triggered)

Integration snippet on the Pi:

```
from ultralytics import YOLO
yolo = YOLO('yolov8n_obstacles.pt')

results = yolo(frame, verbose=False)
boxes = results[0].boxes
if len(boxes) > 0:
    x1, y1, x2, y2 = boxes[0].xyxy[0]
    box_width = (x2 - x1) / frame.shape[1]
    if box_width > 0.30:
        safe_stop()   # obstacle too close
```

12.6 Why YOLOv8n for Edge Deployment

- Model Size: YOLOv8n weights are approximately 6.2 MB — small enough to load quickly on the Pi.
- Inference Speed: On RPi 4B (ARM Cortex-A72), YOLOv8n achieves approximately 200–400 ms per frame in CPU-only mode. Combined with the 50–100 ms ONNX

steering inference, total loop latency is ~300–500 ms, giving ~2–3 FPS obstacle awareness — sufficient for the arena's low speed.

- COCO Pretraining: The yolov8n.pt base weights include general object classes, providing good initialisation before fine-tuning on the obstacle dataset.
- Ultralytics API: The unified train/infer API drastically reduces boilerplate and ensures reliable data loading from YOLO-format datasets.

12.7 Two-Model Architecture Summary

Model	Architecture	Task	Output	Deployment Format
Steering Model	NVIDIA Dave-2 CNN	Predict steering angle from image	Float [-1, +1]	ONNX (.onnx)
Obstacle Detector	YOLOv8n	Detect obstacles in frame	Bounding boxes + confidence	PyTorch (.pt)

Both models run sequentially on the same camera frame. The obstacle detector acts as a safety override layer on top of the behavioral cloning steering output, ensuring the vehicle can react to unexpected objects in the arena without relying on the steering model alone.

13. References

1. Bojarski, M. et al. (2016). End to End Learning for Self-Driving Cars. NVIDIA Technical Report. arXiv:1604.07316.
2. Udacity Self-Driving Car Behavioural Cloning Dataset (Jungle Track). Available on Kaggle: https://www.kaggle.com/datasets/andy8744/udacity-self-driving-car-behavioural-cloning
3. ONNX Runtime Documentation. https://onnxruntime.ai/
4. pigpio Library Documentation. http://abyz.me.uk/rpi/pigpio/
5. tf2onnx: Convert TensorFlow models to ONNX. https://github.com/onnx/tensorflow-onnx
6. Jocher, G. et al. (2023). Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics
7. Udacity Self-Driving Car Obstacles Dataset. Kaggle: https://www.kaggle.com/datasets/barkataliarbab/udacity-self-driving-car-obstacles-dataset